

FORGOT PASSWORD, FRONTEND (direct reset, one page, two steps)

Angular 16.2, Material, Midnight theme. **Replaces the earlier e-mail-link pages.** One component does everything; the `reset-password` component is deleted.

0. What the user sees

1. Login page: **Forgot password?** opens `/forgot-password`.
2. **Step 1:** type the e-mail, **Continue** (disabled until it is a valid e-mail).
3. **Step 2:** "Choose a new password" for that e-mail (a **Change** link goes back). The same strength checklist as registration shows live. If the server asks for it (`reset.require-mobile=true`) there is also a mobile number box.
4. **Change password**, then "Password changed" with a **Log in now** button. The new password works at once.

1. The ideas, from zero (tied to your code)

One component, a step variable. `ForgotPasswordComponent` keeps `step: 'email' | 'password' | 'done'`. The template shows one of three blocks with `*ngIf`. There are two reactive forms: `emailForm` (just the e-mail) and `passForm` (mobile, password, confirm). Reactive forms are the same tool your registration page uses.

Same password rules as registration. `PASSWORD_RULES` is exported from `registration.component.ts`. The `strong` validator checks every rule, and the checklist under the box turns each line green as it is met. The group validator `mismatch` compares the two boxes.

Step 1 asks the server. `PasswordResetService.start(email)` posts to `/api/forgot-password` and gets `{requireMobile}`. The component then adds or removes the validators of the `mobile` control (`setValidators`, `updateValueAndValidity`) so the box is only required when it is shown.

Step 2 saves. `PasswordResetService.reset(email, newPassword, mobile?)` posts to `/api/reset-password`. Answers: 200 shows "done"; 404 shows the message and a "Create an account" link (`#registerFromReset`); 400 shows the server message (weak password, admin, wrong mobile); 429 starts a countdown from the `Retry-After` header and disables the buttons.

Old links. `auth.module.ts` has `{ path: 'reset-password', redirectTo: 'forgot-password' }`, so an old e-mailed link lands on the new page.

2. What goes where

File	Action
<code>services/password-reset.service.ts</code>	replace

File	Action
components/forgot-password/forgot-password.component.ts / .html / .css	replace
modules/auth/auth.module.ts	replace
components/reset-password/ (whole folder)	delete
components/login/login.component.html	the "Forgot password?" link (already in the merged project)
constant.ts	keep FORGOT_PASSWORD: '/api/forgot-password' and RESET_PASSWORD: '/api/reset-password'

3. Check it

1. `ng serve --port 8081`, backend on 8080.
2. Login page, **Forgot password?**, type a registered e-mail, **Continue**.
3. Try a weak password: the button stays disabled and the checklist shows what is missing. Try two different passwords: disabled.
4. Choose a strong password, **Change password**, **Log in now**, log in with it.

Browser test against a mock backend: **29 checks, 0 failures** (link on login, disabled states, step 2 and the Change link, rules, mismatch, success, old password rejected, new password logs in, unknown e-mail, admin refused, old `/reset-password` link, mobile mode right and wrong, phone width, no page errors). The real backend was not part of that test.

4. Complete code

`angularapp/src/app/services/password-reset.service.ts`

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';

/** Answer of step 1. requireMobile = step 2 must also ask for the registered mobile number. */
export interface ForgotStartResponse {
  requireMobile: boolean;
}

/** Forgot password without e-mail: step 1 checks the e-mail, step 2 sets the new password. Both calls are public. */
@Injectable({
  providedIn: 'root'
})
export class PasswordResetService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  start(email: string): Observable<ForgotStartResponse> {
    return this.http.post<ForgotStartResponse>(`${this.apiUrl}${API.FORGOT_PASSWORD}`, { email });
  }

  reset(email: string, newPassword: string, mobile?: string): Observable<{ message: string }> {
    return this.http.post<{ message: string }>(`${this.apiUrl}${API.RESET_PASSWORD}`, { email, newPassword, mobile: mobile || null });
  }
}
```

```
}  
}
```

angularapp/src/app/modules/auth/auth.module.ts

```
import { NgModule } from '@angular/core';  
import { RouterModule, Routes } from '@angular/router';  
import { SharedModule } from '../../../shared/shared.module';  
import { LoginComponent } from '../../../components/login/login.component';  
import { RegistrationComponent } from '../../../components/registration/registration.component';  
import { ForgotPasswordComponent } from '../../../components/forgot-password/forgot-password.component';  
  
const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  { path: 'registration', component: RegistrationComponent },  
  { path: 'forgot-password', component: ForgotPasswordComponent },  
  { path: 'reset-password', redirectTo: 'forgot-password' } // old e-mailed links now land on the new  
page  
];  
  
/** Login, registration and forgot / reset password. Lazy loaded: nobody needs this code until they open  
the login page. @author Suriya */  
@NgModule({  
  declarations: [LoginComponent, RegistrationComponent, ForgotPasswordComponent],  
  imports: [SharedModule, RouterModule.forChild(routes)]  
})  
export class AuthModule {}
```

angularapp/src/app/components/forgot-password/forgot-password.component.ts

```
import { Component, OnDestroy } from '@angular/core';  
import { AbstractControl, FormBuilder, ValidationErrors, Validators } from '@angular/forms';  
import { PasswordResetService } from '../../../services/password-reset.service';  
import { PASSWORD_RULES } from '../../../registration/registration.component';  
  
function strong(c: AbstractControl): ValidationErrors | null {  
  const v: string = c.value || '';  
  return v && PASSWORD_RULES.some(r => !r.test(v)) ? { weak: true } : null;  
}  
  
/**  
 * "Forgot password" without e-mail. Step 1: the user types the e-mail. Step 2: the user types a new  
password (and, only when  
 * the server asks for it, the registered mobile number). The password is changed at once and the user can  
log in with it.  
 */  
@Component({  
  selector: 'app-forgot-password',  
  templateUrl: './forgot-password.component.html',  
  styleUrls: ['./forgot-password.component.css']  
})  
export class ForgotPasswordComponent implements OnDestroy {  
  step: 'email' | 'password' | 'done' = 'email';  
  emailForm = this.fb.group({ email: ['', [Validators.required, Validators.email]] });  
  passForm = this.fb.group({  
    mobile: [''],  
    password: ['', [Validators.required, strong]],  
    confirmPassword: ['', Validators.required]  
  }, { validators: (g: AbstractControl): ValidationErrors | null => g.get('password')?.value ===  
g.get('confirmPassword')?.value ? null : { mismatch: true } });  
  email = '';  
  requireMobile = false;  
  showPassword = false;  
  busy = false;  
  error = '';  
  notRegistered = false;  
  secondsLeft = 0;  
  private timer?: any;
```

```

constructor(private fb: FormBuilder, private resetApi: PasswordResetService) {}

ngOnDestroy(): void { clearInterval(this.timer); }

get emailCtl() { return this.emailForm.get('email'); }
get f() { return this.passForm.controls; }

get checks() {
  const v: string = this.f['password'].value || '';
  return PASSWORD_RULES.map(r => ({ label: r.label, ok: r.test(v) }));
}

/** Step 1 -> step 2. */
next(): void {
  if (this.emailForm.invalid || this.busy || this.secondsLeft > 0) { this.emailForm.markAllAsTouched();
return; }
  this.busy = true;
  this.error = '';
  this.resetApi.start(String(this.emailForm.value.email).trim()).subscribe({
    next: r => {
      this.busy = false;
      this.email = String(this.emailForm.value.email).trim().toLowerCase();
      this.requireMobile = !!r.requireMobile;
      const mobile = this.f['mobile'];
      mobile.setValidators(this.requireMobile ? [Validators.required, Validators.pattern(/^[0-9]{10}$/)]
: []);
      mobile.updateValueAndValidity();
      this.step = 'password';
    },
    error: err => this.fail(err, 'Could not continue. Please try again in a moment.')
  });
}

/** Step 2: change the password. */
save(): void {
  if (this.passForm.invalid || this.busy || this.secondsLeft > 0) { this.passForm.markAllAsTouched();
return; }
  this.busy = true;
  this.error = '';
  this.notRegistered = false;
  const mobile = this.requireMobile ? String(this.f['mobile'].value || '').trim() : undefined;
  this.resetApi.reset(this.email, String(this.f['password'].value), mobile).subscribe({
    next: () => { this.busy = false; this.step = 'done'; },
    error: err => {
      this.notRegistered = err.status === 404;
      this.fail(err, 'The password could not be changed. Please try again in a moment.');
```

```
}
}
```

angularapp/src/app/components/forgot-password/forgot-password.component.html

```
<div class="page-wrap">
  <div class="form-card">
    <div class="icon"><mat-icon>{{ step === 'done' ? 'verified' : step === 'password' ? 'password' :
'lock_reset' }}</mat-icon></div>

    <!-- step 1: e-mail -->
    <ng-container *ngIf="step === 'email'">
      <h2>Forgot your password?</h2>
      <p class="lead">Enter the e-mail you registered with. Next you will choose a new password.</p>
      <form [formGroup]="emailForm" (ngSubmit)="next()" novalidate>
        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="username" />
          <mat-icon matSuffix>mail</mat-icon>
          <mat-error *ngIf="emailCtl?.errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="emailCtl?.errors?.['email']">Enter a valid email</mat-error>
        </mat-form-field>
        <div class="form-error" role="alert" id="emailError" *ngIf="error"><mat-icon>error_outline</mat-
icon>{{ error }}</div>
        <button mat-flat-button color="primary" type="submit" id="continueBtn"
[disabled]="emailForm.invalid || busy || secondsLeft > 0">
          {{ busy ? 'Checking...' : secondsLeft > 0 ? 'Wait ' + secondsLeft + 's' : 'Continue' }}
        </button>
      </form>
    </ng-container>

    <!-- step 2: new password -->
    <ng-container *ngIf="step === 'password'">
      <h2>Choose a new password</h2>
      <p class="lead">For <strong id="forEmail">{{ email }}</strong>. <button type="button" class="link"
id="changeEmail" (click)="backToEmail()">Change</button></p>
      <form [formGroup]="passForm" (ngSubmit)="save()" novalidate>
        <mat-form-field appearance="outline" *ngIf="requireMobile">
          <mat-label>Registered mobile number</mat-label>
          <input matInput id="mobile" type="tel" inputmode="numeric" maxlength="10"
formControlName="mobile" autocomplete="tel" />
          <mat-icon matSuffix>smartphone</mat-icon>
          <mat-error *ngIf="f['mobile'].errors?.['required']">Mobile number is required</mat-error>
          <mat-error *ngIf="f['mobile'].errors?.['pattern']">Enter the 10 digit mobile number</mat-error>
        </mat-form-field>
        <mat-form-field appearance="outline">
          <mat-label>New password</mat-label>
          <input matInput id="newPassword" [type]="showPassword ? 'text' : 'password'"
formControlName="password" autocomplete="new-password" />
          <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword"
[attr.aria-label]="showPassword ? 'Hide password' : 'Show password'">
            <mat-icon>{{ showPassword ? 'visibility_off' : 'visibility' }}</mat-icon>
          </button>
          <mat-error *ngIf="f['password'].errors?.['required']">Password is required</mat-error>
          <mat-error *ngIf="f['password'].errors?.['weak']">Password does not meet all the rules
below</mat-error>
        </mat-form-field>
        <ul class="pw-rules" id="passwordRules" aria-label="Password rules" *ngIf="f['password'].value">
          <li *ngFor="let c of checks" [class.ok]="c.ok"><mat-icon aria-hidden="true">{{ c.ok ?
'check_circle' : 'radio_button_unchecked' }}</mat-icon>{{ c.label }}</li>
        </ul>
        <mat-form-field appearance="outline">
          <mat-label>Confirm new password</mat-label>
          <input matInput id="confirmNewPassword" [type]="showPassword ? 'text' : 'password'"
formControlName="confirmPassword" autocomplete="new-password" />
          <mat-error *ngIf="f['confirmPassword'].errors?.['required']">Confirm your password</mat-error>
        </mat-form-field>
        <div class="form-error small" *ngIf="passForm.errors?.['mismatch'] && f['confirmPassword'].touched"
role="alert">Passwords do not match</div>
      </form>
    </ng-container>
  </div>
</div>
```

```

    <div class="form-error" role="alert" id="resetError" *ngIf="error"><mat-icon>error_outline</mat-
icon>{{ error }}</div>
    <a *ngIf="notRegistered" class="link-block" id="registerFromReset"
routerLink="/registration">Create an account</a>
    <button mat-flat-button color="primary" type="submit" id="saveNewPassword"
[disabled]="passForm.invalid || busy || secondsLeft > 0">
    {{ busy ? 'Saving...' : secondsLeft > 0 ? 'Wait ' + secondsLeft + 's' : 'Change password' }}
    </button>
  </form>
</ng-container>

<!-- done -->
<ng-container *ngIf="step === 'done'">
  <h2 id="resetDone">Password changed</h2>
  <p class="lead">You can log in with your new password now. Any other device that was logged in has
been signed out.</p>
  <a mat-flat-button color="primary" id="goLogin" routerLink="/login">Log in now</a>
</ng-container>

  <p class="alt" *ngIf="step !== 'done'"><a routerLink="/login">Back to login</a></p>
</div>
</div>

```

angularapp/src/app/components/forgot-password/forgot-password.component.css

```

/* Forgot password (Midnight) */
.page-wrap { display: grid; place-items: start center; padding: 40px 16px; }
.form-card { width: 100%; max-width: 460px; }
.icon { display: grid; place-items: center; width: 52px; height: 52px; margin: 0 0 14px; border-radius:
14px; background: var(--gold-soft); color: var(--gold-text); }
.icon mat-icon { font-size: 28px; width: 28px; height: 28px; }
.alt { margin: 18px 0 0; text-align: center; color: var(--muted); font-size: 14px; }
.link { border: 0; background: none; padding: 0; color: var(--gold-text); font-weight: 700; cursor:
pointer; text-decoration: underline; text-decoration-offset: 3px; }
.link-block { display: inline-block; margin: 0 0 12px; color: var(--gold-text); font-weight: 700; }
.pw-rules { display: grid; gap: 6px; margin: 2px 0 14px; padding: 12px 14px; list-style: none; border-
radius: 10px; background: var(--surface-2); border: 1px solid var(--line); }
.pw-rules li { display: flex; align-items: center; gap: 8px; color: var(--muted); font-size: 13.5px; }
.pw-rules li mat-icon { width: 18px; height: 18px; font-size: 18px; }
.pw-rules li.ok { color: var(--ok); }
button[type=submit] { width: 100%; margin-top: 6px; }

```